

CI/CD Checklist

Your Complete Implementation Guide

by

TECHWORLD WITH NANA

I'm Nana, Co-Founder of TechWorld with Nana.

As a DevOps engineer and educator, I love helping engineers build efficient, streamlined workflows and become more valuable with highly-demanded DevOps and Cloud skills

Through my [YouTube channel](#) and my comprehensive [DevOps bootcamps](#), I've helped **1,000,000s of engineers** master the tools and practices that drive modern software development.



About this Checklist

This CI/CD checklist is your roadmap to **go from a manual, error-prone deployment process to an automated, efficient release workflow.**

This guide addresses all the challenges teams run into when implementing continuous integration & continuous delivery.

Just getting started with CI/CD? Or looking to optimize your existing pipelines?

This checklist will help you implement production best practices to reduce deployment stress, improve code quality, and deliver value to your users faster & more reliably.

Use it as your companion on your CI/CD journey, and remember: *automation is an investment that pays dividends through more stable releases, happier developers, and better software!*

Happy building!

Nana Janashia

Co-Founder TechWorld with Nana

Continuous Integration (CI) Checklist

Version Control Best Practices

- ☐ Implement branching strategy (feature branches, main/master branch)
- ☐ Encourage small, frequent commits rather than large changes
- ☐ Set up branch protection rules to prevent direct pushes to main
- ☐ Require code reviews before merging



Automated Testing for CI

- ☐ Set up automated testing on every commit in feature branches
- ☐ Add functional tests to ensure features work as expected
- ☐ Configure linting and code style enforcement
- ☐ Ensure tests run before allowing merge to main branch
- ☐ Set up notifications for test failures



Integration Process

- ☐ Implement automated merge conflict detection
- ☐ Set up test coverage reporting
- ☐ Configure dependency vulnerability scanning
- ☐ Create automated feedback system for developers



Continuous Delivery (CD) Checklist

Deployment Automation

- ☐ Set up automatic artifact creation (Docker images, packages)
- ☐ Implement automatic versioning/tagging of releases
- ☐ Configure artifact repository storage
- ☐ Automate deployment to development environment



End-To-End Validation

- ☐ Set up automated end-to-end tests in development environment
- ☐ Implement integration tests across services
- ☐ Configure automated security scanning
- ☐ Set up performance baseline testing
- ☐ Implement automated rollback capabilities
- ☐ Create validation gates between environments



Pre-Production Deployment

- ☐ Automate deployment to staging/test environment
- ☐ Set up staging environment to mirror production
- ☐ Configure compliance checks if applicable
- ☐ Set up approval gates if needed before production



Continuous Deployment Checklist

Production Deployment Strategy

- ☐ Implement deployment strategy (Canary, Blue-Green, etc.)
- ☐ Set up progressive rollout capabilities
- ☐ Configure automated traffic shifting
- ☐ Implement feature flags for controlled releases
- ☐ Set up automatic rollback triggers



Monitoring & Observability

- ☐ Set up health checks for deployed applications
- ☐ Implement detailed logging
- ☐ Configure alerting for critical issues
- ☐ Set up performance monitoring
- ☐ Create dashboards for deployment status



Feedback & Improvement

- ☐ Set up feedback collection from production deployments
- ☐ Implement post-deployment verification
- ☐ Create system for tracking deployment success metrics
- ☐ Schedule regular pipeline reviews and improvements
- ☐ Measure and track deployment frequency and lead times



Infrastructure and Tooling

CI/CD Pipeline Infrastructure

- ☐ Select appropriate CI/CD tools (Jenkins, GitHub Actions, GitLab CI, etc.)
- ☐ Set up pipeline as code (Jenkinsfile, GitHub workflow files)
- ☐ Implement infrastructure as code for environments
- ☐ Set up proper access controls and permissions
- ☐ Ensure pipeline scalability for team growth

Advanced CI/CD Considerations

Security Integration

- ☐ Implement SAST (Static Application Security Testing)
- ☐ Set up DAST (Dynamic Application Security Testing)
- ☐ Configure container security scanning
- ☐ Implement dependency vulnerability monitoring
- ☐ Set up compliance validation where needed
- ☐ Configure secrets scanning in code



Optimization

- ☐ Implement caching strategies to speed up builds
- ☐ Set up parallel test execution
- ☐ Configure pipeline analytics to identify bottlenecks
- ☐ Configure self-healing for common environmental issues

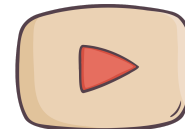


Free Tutorials to Get Started

Start with these free tutorials from us to build your CI/CD foundation:

BEGINNER TUTORIALS

- [GitLab CI/CD Crash Course](#)
- [GitHub Actions Tutorial](#)
- [CI Pipeline with Code Quality Checks](#)
- [Azure DevOps Tutorial](#)
- [Jenkins Pipeline Tutorial | Jenkinsfile explained](#)



ADVANCED TUTORIALS

- [ArgoCD Tutorial - GitOps CD for Kubernetes](#)
- [DevSecOps Tutorial](#)



Take Your Skills to the Professional Level

This checklist tells you **WHAT** to implement, but if you also need the **HOW** - detailed, step-by-step instructions in real-world projects - we've got you covered.

In our comprehensive [DevOps Bootcamp](#) and [DevSecOps Bootcamp](#), you'll learn exactly how to implement everything in this checklist.